Necessary but not sufficient: recoding the ALPN→Kenyon-cell transform restores a category code that the mushroom-body readout does not learn

A FlyWire subcircuit model with growth-matched controls

Nicolas Giavedoni

Revised draft

Abstract

We instantiate a fixed subcircuit of the adult *Drosophila melanogaster* connectome (673 antennal-lobe projection neurons → 384 Kenyon cells → 4 mushroom-body output neurons, with **zero artificial edges**) and ask it to classify 80 four-category prototype patterns. The patterns are seeded, disjoint receptor subsets rather than orthographic or semantic words: the input probe is 1.000 by construction, so the task tests the circuit, not word knowledge. At the physiological operating point the Kenyon-cell population carries no linearly decodable category code (k-fold CV 0.236; held-out 0.255) although the stimulus does (1.000). Raising the gain of the `ALPN → Kenyon cell` transform (a "recode", to 20 or 50, with the calibration rate ceiling widened to 100 Hz) restores the Kenyon-cell code (CV 0.950 / 1.000; held-out 0.958 / 0.997).

The recode is necessary but not sufficient. The online MBON-argmax readout — the only readout that can express what the plastic synapses actually learned — stays at chance in every condition we measured (0.235–0.272; nine conditions, ten seeds). Restoring the Kenyon-cell code therefore does not restore the online learner: the four-dimensional MBON readout and its credit rule are a second, unresolved bottleneck. The post-hoc supervised decoder does rise, but after correction for multiple comparisons the plain trained condition is not significantly above the untrained floor (gain 20: +0.025, Holm $p = 1.0$; gain 50: +0.088, Holm $p = 0.117$).

The largest supervised-decoder numbers come from structural growth (`structural_on` 0.833 at gain 50, 0.608 at gain 20), and growth beats no-growth under this decoder (Holm $p = 0.016$ at both gains). The growth-matched controls demote that result to a **decoder-capacity effect**: removing all reward/label guidance leaves accuracy statistically unchanged (0.785 vs 0.750, Holm $p = 1.0$), breaking category inheritance leaves it unchanged (0.707, Holm $p = 1.0$), and a no-growth decoder given the same 300 extra random Kenyon-cell projections reaches 0.983, above every growth condition (Holm $p = 0.018$). A 4-dimensional readout is the binding constraint; adding feature dimensions, not learning, does the work. Dopamine has no detectable effect on decoded accuracy (`dopamine_off` vs `structural_on` −0.078, Holm $p = 0.34$), but this is a null result about a decoder refit that can undo weight changes, not evidence that plasticity is absent; only the online readout could show that, and it is at chance.

Two further corrections shape the claim. First, the shuffled-label control is judged against chance by a sign-flip permutation test, not against a moved untrained floor. At gain 20 the growth-on shuffled control is above chance (0.330, $p = 0.020$) while the no-growth shuffled control is not (0.230, $p = 0.76$); at gain 50 it is at chance (0.245, $p = 0.56$). This is consistent with growth adding decoder capacity rather than with label leakage, but it is reported as a residual anomaly, not explained away. Second, all numbers here predate a per-condition RNG-isolation fix; under the fixed code `trained` and `structural_off`, which share a configuration, are bit-identical. The archived absolute accuracies are therefore provisional and the paper's conclusions are stated at the level the paired controls support.

The durable contributions are (i) a diagnostic localisation of a class-code bottleneck to the `ALPN → Kenyon cell` transform, (ii) the demonstration that opening that bottleneck is insufficient because the readout does not learn, (iii) released growth-matched controls that demote the paper's own headline to a decoder-capacity effect, and (iv) a reproducible, self-correcting pipeline. The transform result is a calibrated-instance claim, not a claim about the circuit *in vivo*: the gain-1 failure may reflect stimulus drive swamped by spontaneous noise, and the archived runs lack the diagnostics needed to separate gain from signal-to-noise ratio.

1. Introduction

The mushroom body of the insect brain is a canonical substrate for associative learning: sparse, high-dimensional Kenyon-cell representations of odour are read out by mushroom-body output neurons (MBONs) whose synapses are modified by dopaminergic reinforcement neurons (DANs) [Aso et al., 2014; Hige et al., 2015]. Whole-brain connectomes at synapse resolution [Zheng et al., 2018; Scheffer et al., 2020; Dorkenwald et al., 2024; Shiu et al., 2024] now make it possible to ask whether that circuit, instantiated at its measured synaptic structure, can actually *learn a task* under a biologically motivated plasticity rule.

We pose a small, fully specified classification problem: assign 80 seeded four-category prototype patterns (animal, food, tool, place) to four classes. The patterns are not words in any orthographic or semantic sense; each category owns a disjoint subset of input neurons, so the input representation is linearly separable by construction. The problem is small enough to run in hours per condition on a laptop and large enough to have genuine class structure. The scientific question is not whether a larger or better-tuned model can win, but whether the *real wiring* plus a simple, documented rule can learn — and if so, what does the learning.

The study answers with a mechanism and a negative result. The real olfactory chain scores above its untrained floor once the gain of the `ALPN → Kenyon cell` transform is raised; at the physiological operating point the class code is destroyed at that transform. But restoring the code does not restore the online MBON readout, which remains at chance. The recode is therefore necessary, not sufficient: the readout and its credit rule are a second bottleneck that this subcircuit does not overcome. Within the recoded regime structural growth produces the largest supervised-decoder accuracy, but the growth-matched controls show that this is a decoder-capacity effect — it does not require label guidance and is beaten by plain random features. We report the mechanism, the

negative result, an explicit account of how fragile the numbers are, and the controls that demote the paper's own headline.

Section 2 gives background. Section 3 details data, subcircuit, encoding, network, learning rule, recode, evaluation protocol, and the statistical plan. Section 4 reports the result, the probe-based localisation, the growth-versus-dopamine decomposition, the controls, and the anomalies. Section 5 discusses mechanics, threats to validity, and what would change the answer. Section 6 concludes.

2. Background

Connectome. FlyWire release 783 provides a proofread synapse-level wiring diagram of an adult *Drosophila* brain [Dorkenwald et al., 2024; Shiu et al., 2024]. We use it only as a connectivity graph: nodes are neurons, directed weighted edges are synapse counts, and cell classes (e.g. `ALPN`, `Kenyon_Cell`, `MBON`, `DAN`) come from the accompanying annotations [Schlegel et al., 2024].

Mushroom body. Kenyon cells receive predominantly olfactory input from antennal-lobe projection neurons and give sparse representations; MBONs read this population out; DANs carry reinforcement and gate plasticity at the Kenyon-cell → MBON synapses [Aso et al., 2014]. Learning is widely modelled as a three-factor rule: a Hebbian eligibility trace multiplied by a modulatory (reward) signal [Hebb, 1949; Schultz et al., 1997; Gerstner et al., 2018]. Expansion coding predicts that a high-dimensional, sparse Kenyon-cell layer makes odour categories more linearly separable, provided the projection is calibrated so that stimuli are not lost in noise [Litwin-Kumar et al., 2017; Dasgupta et al., 2017]. The insect mushroom body also contains a dense inhibitory feedback neuron, APL, which enforces sparseness [Lin et al., 2014]; our extracted subcircuit does not include it (Section 5.5).

Simulation. We use Brian2 [Stimberg et al., 2019] with leaky integrate-and-fire neurons and a per-run no-stimulus calibration. Every run writes a manifest with the resolved configuration, seed state, data checksums, calibration, subcircuit statistics, codegen target, hardware, and git commit.

3. Methods

3.1 Data and subcircuit extraction

The pipeline verifies and loads the FlyWire proofread connections (`data/flywire/proofread_connections_783.feather`, 16.8 M rows), maps neurons to a stable index, assigns connection signs, and extracts a subcircuit by cell class. Output neurons are four MBONs. The configuration selects the olfactory pathway:

```
ALPN (olfactory input) → Kenyon cell (mushroom body) → MBON (output)
                                      ↑
                                      |
                              DAN (reinforcement)
```

Measured connectivity used (edges / synapses) is ALPN→KC 28,144 / 329,394; KC→MBON 89,315 / 256,719; DAN→KC 49,616 / 60,657. The extracted subcircuit contains **1,065 neurons, 23,075 edges and 120,424 synapses with zero artificial edges** (673 ALPN inputs, 384 Kenyon cells, 4 DAN, 4 MBON). Because a global weight scale is not identifiable a priori, the network is

calibrated once per run to a spontaneous regime; the accepted scale was 4.0 (noise weight 4.0, 6 Hz background).

Subsampling. Stages are capped: the 685 ALPN annotation candidates are ranked by the synapses they receive from the input stage and 673 survive edge extraction; Kenyon cells are capped at 384, DANs at 4, MBONs at 4. The caps are required for tractability and they change the circuit relative to the real animal: real Kenyon cells receive roughly 5–7 projection-neuron inputs, whereas the extracted subcircuit's in-degree distribution is determined by the ranking and the cap, not by biology. We do not claim that the cap preserves the physiological fan-in; the in-degree distribution of the extracted network is recorded in each manifest and should be read before generalising any result.

Cell-type caveat. The four reinforcement neurons are `DAN` cells in this olfactory configuration. (The separate visual configuration uses `MBIN` cells, which comprise two APL GABAergic neurons and two DPM neurons; that labeling does not apply here.) Four DANs are a tiny fraction of the animal's dopaminergic population, and real MBONs are compartmentalised with specific DAN partners rather than four generic output units. Mapping four arbitrary task categories onto four MBONs is therefore an arbitrary modelling choice, not a claim about valence organisation.

3.2 Stimulus encoding

The task assigns four-letter tokens to four categories, but the tokens are **not** processed as letters or meaning. Each category owns a fixed, seeded, disjoint subset of the input neurons; each token drives 64 inputs from its own category prototype plus 4 cross-category inputs at graded drive in [0.35, 1.0], with the rest silent. The stimuli are therefore **category prototype patterns with partial cross-category overlap** — a deliberately easy, low-dimensional task. "Word" below denotes the token that indexes a prototype; no orthographic or semantic structure is present. The dataset has 80 tokens (20 per category); the seeded split is 15 training and 5 test tokens per category (60/20). The mapping uses the selected input-stage size (673 inputs).

3.3 Network and plasticity

The network is a population of LIF neurons wired by the extracted edges. Plasticity is restricted to the Kenyon-cell ⟶ MBON synapses. Each synapse carries a decaying eligibility trace updated by pre/post activity; the weight update is the trace multiplied by a dopamine signal and a learning rate, then clipped. Dopamine is delivered per class: on each trial the four MBONs receive credit according to whether their class is correct, so the rule is a per-class reward. A `dopamine off` switch disables the multiplicative factor while leaving the trace dynamics intact.

Structural plasticity adds a reserve pool of silent, unconnected neurons (1000; 700 available, 300 activated). At fixed intervals, reserve neurons are recruited near the most successful readout unit, inherit its category and at most (1 – divergence) of its inputs, and draw the remainder pseudo-randomly; deletions retire the weakest mature reserve synapses so they track a configured fraction of additions. The mechanism is exploratory — it has no published precedent — and is switched on/off as an ablation. **The recruitment rule is supervised by the network's own predictions and transfers the winning unit's category, so it may construct a class prototype directly**

rather than learn one; Section 5.2 makes this the central threat to the interpretation of the headline number.

The real `DAN → Kenyon cell` edges are extracted but are **not simulated**: `build_network` wires consecutive stage pairs, within-stage recurrence, and the plastic `mushroom body → output` pair. Because `reinforcement` is configured after `mushroom_body`, `reinforcement → mushroom_body` is a backward pair and is dropped; the reward is applied as a global scalar gated by the real DANs' spike counts rather than through the real synapses. The config's "real DAN ⟶ Kenyon cell" pathway is therefore not the one trained.

3.4 Recoding the input transform

The `ALPN → Kenyon cell` projection carries a configurable pair gain (default 1.0, the identity). The recode raises this gain to 20 or 50 and widens the calibration rate ceiling (`calibration.max_mean_rate_hz`, default 15 Hz) to 100 Hz, so that calibration may accept a scale at which the Kenyon cells carry the label (the single global weight scale otherwise falls as the gain rises, holding Kenyon-cell activity roughly constant and nullifying the gain). **The 100 Hz ceiling is far outside the physiological 0.5–15 Hz band**; the recode is a diagnostic intervention, not a claim about how the circuit operates *in vivo*. Because gain and calibration act together, the recode is confounded with signal-to-noise ratio: at gain 1 the stimulus drive may be swamped by the Poisson noise input (weight 4.0) and the spontaneous-rate constraint. Section 5.1 treats this confound explicitly.

3.5 Readout and evaluation

The reported readout is a **supervised linear decoder fitted on a fixed post-training feature pass** — a specified, documented departure from an online MBON readout that isolates *representation* quality from *decoding*. An online MBON-argmax readout (accuracy of the network's own class decision over the training presentations) is also produced. The online readout is the quantity that tests the plasticity rule; the decoder tests the representation. Confusing the two is the main interpretive risk, and we keep them separate throughout.

Representation probes. To separate "the network cannot learn" from "the evaluation cannot detect learning", two linear probes are fitted on a *fresh untrained* network and scored on held-out tokens with the same decoder protocol: one on the input stage and one on the Kenyon-cell stage. A probe reports training, k-fold cross-validation, and held-out accuracy per seed and aggregated. A *positive-control gate* passes only if the input probe decodes above 0.90 and the Kenyon-cell probe above chance + 0.05.

Conditions. The trained condition is nominally the main result. Controls are: untrained; shuffled training labels (`shuffled_labels`); dopamine off (`dopamine_off`); structural plasticity on/off (`structural_on`, `structural_off`); supervised-pretrained readout (`trained_pretrained`); and two isolation controls — `dopamine_off_no_growth` and `shuffled_labels_no_growth` — which complete the dopamine × growth and shuffled × growth 2 × 2 read-outs. Note that **`trained` and `structural_off` are the same configuration** (`{"trained": true, "structural": false}`) and are retained as an internal replicate; under the fixed RNG code below they are bit-identical.

RNG isolation. Each condition and each probe pass begins with `seed_everything(seed)`. Consequently condition order does not affect a condition's random stream, and identical configurations produce identical results. The archived runs analysed here predate this fix and were executed with a shared per-seed stream; their `trained` and `structural_off` difference (0.363 vs 0.370; 0.418 vs 0.435) measures run-order variance, not an effect. The fix was validated on a smoke run where `trained` and `structural_off` are now exactly equal. All archived numbers are therefore reported as provisional, and only paired contrasts within a run are interpreted.

Statistics. One analysis plan is used throughout, at $\alpha = 0.05$.

- *Condition versus chance.* One-sided Wilcoxon signed-rank test of the per-seed accuracies against 0.25, with Holm correction across the condition family in each run. The exact one-sided floor at n = 10 is 0.00098, so "p < 0.001" is attainable only for the one-sided test and "p < 0.002" only for a two-sided test; we never report beyond the exact floor.
- *Paired contrasts.* Two-sided Wilcoxon signed-rank tests on per-seed differences, with Holm correction across a pre-specified 11-pair family (learning, growth, dopamine, decoder capacity, and anomaly contrasts). The exact two-sided floor at n = 10 is 0.00195; values are reported as 0.002.
- *Leakage.* The shuffled-label control is tested by a sign-flip permutation test against chance (exact enumeration up to 16 seeds, Monte Carlo otherwise). A warning is raised when $p < 0.05$. The untrained floor is reported descriptively but is **not** used as the null; comparing against it after the control failed would move the goalpost.
- *Descriptive intervals.* 95 % percentile-bootstrap CIs over seeds are reported for readability, not for inference.
- *Resolution.* There are 20 unique held-out tokens per seed, so a single-token change moves accuracy by 0.05 and $n = 10$ seeds is the inferential unit. The `eval_repeats = 3` presentations are deterministic (`learn=False`) and are not independent test samples.

Diagnostics. Each probe pass now records, per stage, the spontaneous firing rate (no stimulus, rest-length window) and the evoked rate during one presentation of each category prototype, together with the fraction of Kenyon cells with at least one spike in the stimulus window and their mean spike count. These numbers are needed to decide whether a gain-1 failure is a transform failure or a signal-to-noise failure. They are **not available for the archived runs analysed here** and are a required measurement for the re-run.

3.6 Reproducibility

Seeds are independent, so each formal run executes one `evaluate` process per seed (`flyread sweep`), resumes completed (seed, condition) pairs from a checkpoint, and merges the shards into a single report. Kernels were compiled with Cython. All archived runs carry their manifest. The analysis plan, condition list, and contrast family are specified in `configs/default.yaml` and the evaluation module; the commit used for a run is recorded in its manifest.

4. Results

4.1 The untrained network is above chance, so the floor matters

The untrained network scores **0.338** at gain 20 (Holm p = 0.039) and **0.330** at gain 50 (Holm p = 0.049) against the 0.25 chance level (Table 1). This is the recoded representation's own decodability, not learning. Testing the trained condition against chance alone would therefore inflate its significance; the meaningful comparison is the paired contrast against the untrained control.

4.2 Neither gain produces a significant learning effect under corrected statistics

Table 1 gives the two recoded runs (`runs/recode-v2-merged`, gain 20; `runs/recode-50-v2-merged`, gain 50; 10 seeds each). Against chance, most conditions are significant. Against the **untrained floor**, the plain trained condition is not: gain 20 +0.025 (Wilcoxon p = 0.49; Holm p = 1.0) and gain 50 +0.088 (Wilcoxon p = 0.023; Holm p = 0.117). The gain-50 comparison is nominally significant before correction and does not survive it. We therefore do not claim that the dopamine-gated rule learns in this regime.

Table 1: **Table 1.** Supervised-decoder held-out accuracy by condition (10 seeds, chance 0.25), 95 % percentile-bootstrap CIs over seeds, and Holm-corrected one-sided Wilcoxon p against chance within each run. Untrained floor in bold. The online MBON-argmax readout was not recorded in these two runs; it is reported for the control run in Table 3. `trained` and `structural_off` share a configuration.

condition	gain 20	95 % CI	p (Holm)	gain 50	95 % CI	p (Holm)
untrained (internal floor)	**0.338**	[0.277, 0.393]	0.039	**0.330**	[0.263, 0.390]	0.049
trained	0.363	[0.307, 0.410]	0.024	0.418	[0.367, 0.467]	0.008
structural_off (≡ trained config)	0.370	[0.312, 0.415]	0.024	0.435	[0.395, 0.477]	0.008
shuffled_labels	0.330	[0.272, 0.388]	0.039	0.245	[0.195, 0.303]	0.60
dopamine_off	0.597	[0.532, 0.658]	0.008	0.785	[0.708, 0.850]	0.008
dopamine_off_no_growth	0.357	[0.298, 0.415]	0.024	0.387	[0.333, 0.435]	0.008
trained_pretrained	0.512	[0.470, 0.555]	0.008	0.340	[0.300, 0.380]	0.008
structural_on	**0.608**	[0.552, 0.663]	0.008	**0.833**	[0.785, 0.878]	0.008

Table 2: **Table 2.** Paired per-seed contrasts for the main runs (two-sided Wilcoxon, Holm-corrected across the pre-specified 11-pair family; values at the exact floor are shown as 0.002). Positive Δ means the first condition is higher.

contrast	Δ (gain 20)	p (Holm)	Δ (gain 50)	p (Holm)
trained – untrained	+0.025	1.0	+0.088	0.117
structural_on – structural_off	+0.238	0.016	+0.398	0.016
structural_on – trained	+0.245	0.027	+0.415	0.016
structural_on – dopamine_off	+0.012	1.0	+0.048	0.59
dopamine_off – dopamine_off_no_growth	+0.240	0.029	+0.398	0.016
trained – structural_off	−0.007	1.0	−0.017	0.91
trained_pretrained – untrained	+0.173	0.027	+0.010	0.98
shuffled_labels – untrained	−0.008	1.0	−0.085	0.52

4.3 Growth beats no-growth under the supervised decoder

The growth-on conditions carry the large decoder accuracies: `structural_on` 0.608 at gain 20 and 0.833 at gain 50, versus 0.370 and 0.435 for the identical `structural_off`/`trained` configuration. The paired growth effect is +0.238 (Holm p = 0.016) and +0.398 (Holm p = 0.016). The same gap appears within the dopamine-off arm (`dopamine_off` − `dopamine_off_no_growth`: +0.240 and +0.398, Holm p = 0.029 / 0.016). Structural plasticity executes as designed: 300 reserve neurons are activated and ≈ 13.2–13.9 k synapses are created (150 retired) per seed, bringing the network from 1,065 to 1,365 neurons.

Dopamine does not move decoded accuracy: within the growth-on regime the contrast is +0.012 at gain 20 (Holm p = 1.0) and +0.048 at gain 50 (Holm p = 0.59). This is a **null result** (n = 10), not evidence that dopamine does nothing: the supervised decoder is refitted after training and can undo any weight change, so it is largely blind to the plasticity rule. The online readout, which can see the learned weights, is at chance (Section 4.5). The correct statement is that we detect no effect of the dopamine switch on decoded accuracy.

4.4 The recode restores the Kenyon-cell code but not the online readout

The representation probes locate the physiological bottleneck (Table 5). At gain 1 the stimulus decodes perfectly (input probe 1.000) but the Kenyon-cell population does not (CV 0.236; held-out 0.255; `runs/pctrl-v1-merged`, `runs/pctrl-sep-merged`), and a supervised-pretrained readout

that fits the Kenyon-cell features on the training set still generalises at chance. Raising the gain restores the code: Kenyon-cell CV decodability rises to **0.950** at gain 20 (held-out 0.958) and **1.000** at gain 50 (held-out 0.997).

Table 3: **Table 5.** Kenyon-cell representation probe versus the `ALPN → Kenyon cell` gain. CV and held-out columns are separate quantities (the earlier draft mixed them). Gain-1 values are from the physiological reference runs; gain-20/50 values are from the recode runs' untrained networks.

ALPN → Kenyon gain	input CV	KC CV	KC held-out	trained decoder
1 (physiological)	1.000	0.236	0.255	0.262
20	1.000	0.950	0.958	0.363
50	1.000	1.000	0.997	0.418

The code is restored; the learner is not. In the growth-matched control run, where the online argmax was recorded for every condition, it stays at chance (0.235–0.272; Table 3) even at gain 50 where the Kenyon-cell probe is 1.000. The recode is thus **necessary but not sufficient**: it opens the representational bottleneck, but the four-dimensional MBON argmax and its credit rule remain a second bottleneck. This is the central correction to the earlier draft, which claimed that the network "then learns".

4.5 Growth-matched controls demote growth to decoder capacity

The `structural_on` gain had three possible explanations: the recruitment rule learns a class code; it merely expands the decoder's feature budget; or its label-guided category inheritance constructs a class prototype. The controls in `configs/odor-recode-50-controls.yaml` separate them, run at 10 seeds (`runs/recode-50-controls-merged`, gain 50, 600 trials): growth with random recruit categories, growth with no reward/label guidance at all, and a no-growth decoder given the same number of extra feature dimensions as growth adds (300 fixed random projections of the Kenyon-cell population).

Table 4: **Table 3.** Growth-matched control run (`runs/recode-50-controls-merged`, gain 50, 10 seeds, chance 0.25, 95 % bootstrap CIs over seeds). The internally-paired contrasts within this run are the quantities of interest; absolute numbers differ from the main recode run because the archived runs share a per-condition RNG stream (Section 3.5).

condition	decoder acc (95 % CI)	online argmax
untrained	0.330 [0.263, 0.392]	0.250
trained	0.418 [0.365, 0.467]	0.235
structural_off	0.378 [0.340, 0.417]	0.260
structural_on	**0.750** [0.672, 0.828]	0.250
structural_random_category (control)	0.707 [0.603, 0.802]	0.272
structural_unguided (control)	0.785 [0.693, 0.867]	0.262
dopamine_off	0.828 [0.722, 0.907]	0.250
dopamine_off_no_growth	0.432 [0.363, 0.493]	0.250
trained_extra_features (control)	**0.983** [0.973, 0.992]	0.248

Table 5: **Table 4.** Paired per-seed contrasts for the control run (two-sided Wilcoxon, Holm-corrected across the same 11-pair family). Positive Δ means the first condition is higher.

contrast	Δ accuracy	p (Holm)	reading
structural_on – structural_off	+0.372	0.018	growth beats no growth
structural_on – trained	+0.332	0.018	growth beats plain training
structural_on – structural_unguided	−0.035	1.0	label guidance not needed
structural_on – structural_random_category	+0.043	1.0	category inheritance not needed
structural_on – dopamine_off	−0.078	0.34	dopamine off is not worse
trained – trained_extra_features	−0.565	0.018	extra decoder capacity dominates
trained – untrained	+0.088	0.117	no significant training increment
trained – structural_off	+0.040	0.71	identical configs differ by RNG order
dopamine_off – dopamine_off_no_growth	+0.397	0.023	growth effect within dopamine-off

Three conclusions:

- **Growth beats no-growth under the supervised decoder** (`structural_on` 0.750 vs `structural_off` 0.378 and `trained` 0.418; +0.37, Holm $p = 0.018$).
- **Label guidance is not the mechanism.** Removing all reward/label guidance (`structural_unguided` 0.785) or breaking category inheritance (`structural_random_category` 0.707) leaves accuracy statistically unchanged (Holm $p = 1.0$ for both). The growth benefit is therefore not the label-guided prototype construction we flagged as the central risk.
- **Decoder capacity explains it.** A **no-growth** decoder given 300 random Kenyon-cell projections reaches **0.983** — above every growth condition and 0.565 above `trained` (Holm $p = 0.018$). Since the Kenyon-cell probe at gain 50 is ≈ 1.000, this is close to the best a linear decoder can do with 300 features; growth likewise just adds 300 feature dimensions. The honest reading is that growth improves *access* to an already decodable Kenyon-cell code, not that it learns one.

The controls were designed with the expectation that `structural_on` would beat the unguided/random-category controls and that `trained_extra_features` would stay near the floor. Both predictions failed, and we report that failure as the result. The online argmax stays at chance (0.235–0.272) in every condition.

4.6 The above-chance shuffled control is a residual anomaly, not exculpated

Under the pre-specified leakage rule (sign-flip permutation against chance), the growth-on shuffled-label control is above chance at gain 20: mean 0.330, $p = 0.020$ (Table 1); at gain 50 it is at chance (0.245, $p = 0.56$). The corresponding no-growth isolation controls are at or below chance: 0.230 at gain 20 ($p = 0.76$, `runs/recode-v2-extra-merged`) and 0.233 at gain 50 ($p = 0.73$, `runs/recode-50-extra-merged`). The earlier draft excused the gain-20 value by raising the leakage threshold to the untrained floor after seeing the control fail; that is exactly the goalpost move the permutation test is designed to prevent, and we no longer do it.

The pattern is consistent with the decoder-capacity account: when growth is on, the network's own predictions determine recruitment and the recruited units add decoder features, so even a shuffled-label network can become generically decodable; when growth is off, the control is at chance. It is not consistent with label leakage, because the decoder is fitted on the labels each trial was presented with (Section 4.7) and the no-growth control is at chance. But we do not have a direct measurement isolating the mechanism at gain 20, so we report the anomaly and flag the gain-20 run as the one that requires the RNG-fixed re-run before its shuffled control can be considered resolved.

4.7 A decoder-fit leak, caught and fixed

An earlier recode attempt (`recode-v1`) produced a trained accuracy of 0.363 but its shuffled-label control scored **0.615**, raising the leakage warning and invalidating the run. The cause was a harness bug: the supervised decoder was fitted on the dataset's (true) labels in *every* condition, including the shuffled control, so the control had a classifier reading true labels off the network's training outputs — a mapping the network was never taught. The fix fits the decoder on the labels each trial was actually *presented* with. This is a no-op for every non-shuffled condition (there the presented labels are the dataset labels), so all other numbers stand; the shuffled control dropped to the untrained floor. A regression test reproduces the leak and guards against it. This is a methods note about the harness, not a scientific contribution.

4.8 The pretrained-readout anomaly

`trained_pretrained` is 0.512 at gain 20 and 0.340 at gain 50, despite the Kenyon-cell code being *better* at gain 50 (Table 5). The contrast against untrained is +0.173 (Holm $p = 0.027$) at gain 20 and +0.010 (Holm $p = 0.98$) at gain 50. The earlier draft also reported an online argmax of 0.351 for this condition; **that number is not present in any archived result and is withdrawn.** The condition is under-specified (the pretraining budget, the features it is fitted on, and the interaction with gain are not matched across runs), and we do not have an explanation. We report it as an anomaly rather than narrating it away.

5. Discussion

5.1 The input transform is necessary, and confounded with signal-to-noise

At the physiological operating point the category code is not linearly decodable from the Kenyon-cell population (CV 0.236), while the input is perfectly decodable (1.000). Raising the `ALPN → Kenyon cell` gain restores the code (0.950 ⟶ 1.000 CV). Within this calibrated regime, the transform is a bottleneck: no readout we tried recovers the class code at gain 1, and the single intervention that restores it is the one that raises the decoded accuracy. This is the study's most robust localisation.

It is not, however, a clean mechanistic claim. Gain and calibration move together, and at gain 1 the evoked Kenyon-cell drive may simply be swamped by the Poisson noise input (weight 4.0), the spontaneous-rate constraint, and Kenyon-cell recurrence. If so, the gain-1 failure is a **signal-to-noise failure at the transform**, not evidence that the transform computes the wrong thing. The diagnostics needed to distinguish these — spontaneous versus evoked Kenyon-cell rates and per-odour sparseness — were added to the code during revision but are not present in the archived runs. Until the re-run, the correct statement is that the transform is *necessary in this calibrated regime*, not that it is *the* mechanism in the animal. The same caution applies to the 100 Hz calibration ceiling, which is far outside the physiological 0.5–15 Hz band.

The sensitivity to projection gain and sparsity is qualitatively what expansion coding predicts [Litwin-Kumar et al., 2017; Dasgupta et al., 2017], but we have not measured Kenyon-cell sparseness in the archived runs, and the subcircuit omits APL, the inhibitory feedback that enforces sparseness in the real mushroom body [Lin et al., 2014]. The result is therefore a calibrated instance of an expansion-coding intuition, not a test of it.

5.2 What the largest number rests on: decoder capacity

The best result (0.833 main run; 0.750 matched-control run) is produced by the growth-on conditions, and the structural-recruitment rule is a **supervised, exploratory** mechanism: a reserve neuron is recruited near the most successful readout unit and *inherits that unit's category* and most of its inputs, and the recruited units become extra decoder feature dimensions. The natural worry was that the rule constructs a class prototype rather than learning a code. The controls settle it, and they go against the growth interpretation:

- The gain is **unchanged when all label guidance is removed** (`structural_unguided` 0.785 vs `structural_on` 0.750, Holm p = 1.0) and when category inheritance is broken (`structural_random_category` 0.707, Holm p = 1.0), so label-guided prototype construction is not the mechanism.
- A **no-growth decoder given 300 random Kenyon-cell projections reaches 0.983** (Holm p = 0.018 vs `trained`), above every growth condition. Growth adds the same number of feature dimensions, so its benefit is expanded decoder access to a Kenyon-cell code the recode already made decodable — a **decoder-capacity / feature-expansion effect**, not a learned structure.

Because the Kenyon-cell probe is ≈ 1.000, 0.983 is close to the ceiling a 300-feature linear decoder should reach; it is a sanity check on capacity, not an independent discovery.
- The online MBON-argmax readout never leaves chance, so growth adds decoder-readable structure without producing an online class decision.

The honest summary is narrower than a learning claim: under the recode the Kenyon-cell population carries a class code that an offline supervised decoder can exploit, and structural growth is one — sub-optimal — way to give that decoder more to work with. The plain dopamine-gated trained condition is not significantly above the untrained floor, and dopamine has no detectable effect on decoded accuracy. The durable positive result is the localisation of the gate to the `ALPN → Kenyon cell` transform, together with the negative result that opening it does not make the online readout learn.

5.3 The readout bottleneck is the unresolved half of the story

The recode restores a code that the supervised decoder reads at 0.833, yet the online MBON-argmax readout is at chance in all nine control conditions. This is the result the earlier draft under-weighted. Two possibilities remain: the four-dimensional MBON readout is too small to express four categories robustly, or the per-class credit rule does not move the weights in a useful direction (or both). The decoder-capacity control (300 random features ⟶ 0.983) shows that the feature budget is sufficient; the online readout's failure therefore points at the credit rule or at the four-MBON bottleneck, not at the representation. We did not run the RPE-gated per-class variant on the main sweep, so we do not claim that dopamine is the culprit. We claim that the recode is necessary and that the readout is a second, unresolved failure.

5.4 Dopamine: no detectable effect, not "nothing"

Within the growth-on regime, switching dopamine off changes decoded accuracy by +0.048 at gain 50 (Holm $p = 0.59$) and +0.012 at gain 20 (Holm $p = 1.0$). The earlier draft described this as dopamine "changing nothing"; with $n = 10$ and a decoder refit, the correct statement is that we detect no effect. The decoder is fitted after training and can absorb any weight change into a new readout, so it is a weak instrument for the plasticity rule. The online argmax is the right instrument, and it is at chance. A null on the decoder is not evidence that the three-factor rule is inert.

5.5 Biological scope and caveats

- **Arbitrary category-to-MBON mapping.** Four task categories are mapped onto four MBONs. Real MBONs are compartmentalised, receive valence-specific DAN input, and are not generic category units; the mapping is a modelling choice, not a claim about mushroom-body organisation.
- **No APL, no measured sparseness.** The extracted subcircuit omits APL, and the archived runs did not report Kenyon-cell activity sparseness. The 100 Hz calibration ceiling is noted but not converted into a physiological claim.
- **Subsampling changes fan-in.** Capping to 384 Kenyon cells and 673 ALPNs changes the in-degree distribution relative to real Kenyon cells (~5–7 PN inputs). Results are conditional on the extraction.

- **DAN edges are not simulated.** The real `DAN → Kenyon cell` synapses are extracted but dropped as a backward stage pair; reward is a global scalar gated by DAN spike counts.
- **Visual-pathway statement corrected.** The earlier draft implied the mushroom body is not a visual target. Visual projection neurons do innervate the mushroom-body calyx [Vogt et al., 2016; Li et al., 2020]. The correct statement is narrower: this configuration defines no direct optic-lobe ⟶ Kenyon-cell stage, so the visual chain is not the one modelled here.

5.6 Statistics and archival caveat

All numbers in this paper come from archived runs that predate the RNG-isolation fix. Under the fix, conditions start from their own seeded stream, so `trained` and `structural_off` are bit-identical and condition order cannot change a result. The archived `trained` vs `structural_off` differences (0.007 at gain 20, 0.017 at gain 50) and the `structural_on` difference between runs (0.833 vs 0.750) therefore include run-order noise. We have not re-run the full sweep (≈ 24 h per seed single-core; the archived sweeps took ≈ 126 h and ≈ 242 h of single-core compute). The paper is written so that the conclusions rest on internally paired contrasts and on effects far larger than the run-order variation: the growth-within-run contrast is +0.37–0.40, the decoder-capacity contrast is +0.57, and the online-readout null holds across all nine conditions. Absolute accuracies should be treated as provisional until the RNG-fixed re-run, which is the first item of future work.

5.7 Limitations

- Task and network are small by design (CPU-only reproducibility); the stimuli are low-dimensional prototype patterns, so the results speak to this calibrated regime and this rule, not to mushroom-body or word learning in general.
- The transform localisation is confounded with signal-to-noise ratio; the separating diagnostics are implemented but not yet measured in a full run.
- The growth number is explained by decoder capacity: the growth-matched controls show the gain is unchanged without label guidance and is exceeded by random features with no growth, so the largest numbers are not evidence of circuit learning.
- Only internally-paired contrasts are interpreted; absolute numbers are provisional pending the RNG-fixed re-run.
- The reported readout is a supervised post-training decoder; the online readout is at chance; the recode leaves the physiological band.
- All conditions share a single calibration per run; the recode is reported at two gains, so a monotone gain dependence is not established by more than two points.

6. Conclusion

A real, synapse-level FlyWire subcircuit with zero artificial edges carries a linearly decodable category code once a single input transform is recoded: raising the `ALPN → Kenyon cell` gain restores a Kenyon-cell code that the physiological operating point destroys (input probe 1.000; Kenyon-cell CV 0.236 ⟶ 0.950–1.000). The recode is **necessary but not sufficient**. The online MBON-argmax readout is at chance in every condition we measured (0.235–0.272 across nine conditions), so the four-dimensional readout and its credit rule remain a second, unresolved

bottleneck. The post-hoc supervised decoder rises with gain, but the plain trained condition is not significantly above the untrained floor after correction (Holm $p = 1.0$ at gain 20; 0.117 at gain 50).

The largest supervised-decoder numbers come from structural growth (0.833 / 0.750, beating growth-off by +0.37–0.40). The growth-matched controls show what those numbers are: the gain is unchanged when all label guidance is removed (Holm $p = 1.0$) or category inheritance is broken (Holm $p = 1.0$), and it is exceeded by a no-growth decoder given 300 random Kenyon-cell projections (0.983, Holm $p = 0.018$). The growth effect is a decoder-capacity effect — expanded access to an already decodable representation — not a learned, label-guided prototype, and not an online learned behaviour.

We also report the failures and anomalies that constrain the claim: a gain-20 shuffled control above chance that the earlier draft explained away by moving the leakage threshold; a pretrained-readout anomaly (0.512 $\longrightarrow$ 0.340) with a previously reported online value that could not be reproduced and is withdrawn; and an RNG-isolation bug that makes all archived absolute numbers provisional. The durable contributions are (i) a diagnostic localisation of a class-code bottleneck to the `ALPN → Kenyon cell` transform, (ii) the negative result that opening that bottleneck does not make the online readout learn, (iii) released growth-matched controls that demote the paper's own headline, and (iv) a reproducible, self-correcting pipeline in which all of these claims can be inspected and challenged.

Data and code availability

Code: `ngiaved/fly-word-categorization` (FlyWire data are CC-BY-NC and downloaded by `flyread fetch-data`). The archived run directories (`runs/recode-v2-merged`, gain 20; `runs/recode-50-v2-merged`, gain 50; `runs/recode-50-controls-merged`; `runs/recode-v2-extra-merged`, `runs/recode-50-extra-merged`; and the physiological references `runs/odor-v1-merged`, `runs/pctrl-v1-merged`, `runs/pctrl-sep-merged`) are **not committed to the repository** because `runs/` is gitignored and the raw artefacts are large. They are being prepared for deposit on Zenodo; the DOI will be inserted here when assigned. `CITATION.cff` gives the citation metadata. Every archived run carries its manifest (config, seed state, checksums, calibration, subcircuit statistics, codegen target, hardware, git commit). The growth-matched control configuration is released at `configs/odor-recode-50-controls.yaml`. The revised statistics are produced by `src/flyread/evaluate.py` (Wilcoxon + Holm, sign-flip leakage test, paired contrasts) and can be recomputed from each run's `per_seed_test_accuracy` block.

References

1. Aso Y, Hattori D, Yu Y, et al. (2014). The neuronal architecture of the mushroom body provides a logic for associative learning. *eLife* 3:e04577.
2. Dasgupta S, Stevens CF, Navlakha S (2017). A neural algorithm for a fundamental computing problem. *Science* 358(6364):793–796.
3. Dorkenwald S, Matsliah A, Sterling AR, et al. (2024). Neuronal wiring diagram of an adult brain. *Nature* 634:124–138.

4. Gerstner W, Lehmann M, Liakoni V, Corneil D, Brea J (2018). Eligibility traces and plasticity: a theoretical framework. *Frontiers in Neural Circuits* 12:53.
5. Hebb DO (1949). *The Organization of Behavior*. Wiley.
6. Hige T, Aso Y, Rubin GM, Turner GC (2015). Plasticity-driven individualization of olfactory coding in mushroom body output neurons. *Nature* 526(7572):258–262.
7. Lappalainen JK, Tschopp FD, Prakhya S, et al. (2024). Connectome-constrained networks predict neural responses across the fly visual system at single-neuron resolution. *Cell* 187(5):1145–1161.
8. Li F, Lindsey JW, Marin EC, et al. (2020). The connectome of the adult *Drosophila* mushroom body provides insights into function. *eLife* 9:e62576.
9. Lin AC, Bygrave AM, de Calignon A, Lee T, Miesenböck G (2014). Sparse, decorrelated odor coding in the mushroom body enhances learned odor discrimination. *Nature Neuroscience* 17(4):559–568.
10. Litwin-Kumar A, Harris KD, Axel R, Sompolinsky H, Abbott LF (2017). Optimal degrees of synaptic connectivity. *Neuron* 93(5):1153–1164.
11. Scheffer LK, Xu CS, Januszewski M, et al. (2020). A connectome and analysis of the adult *Drosophila* central brain. *eLife* 9:e57443.
12. Schlegel P, Yin Y, Bates AS, et al. (2024). Whole-brain annotation and multi-connectome cell typing of *Drosophila*. *Nature* 634:139–152.
13. Schultz W, Dayan P, Montague PR (1997). A neural substrate of prediction and reward. *Science* 275:1593–1599.
14. Shiu PK, Sterne GR, Spiller N, et al. (2024). A Drosophila brain connectome at single-neuron resolution. *Nature* 634:210–217.
15. Stimberg M, Brette R, Goodman DFM (2019). Brian 2, an intuitive and efficient neural simulator. *eLife* 8:e47314.
16. Vogt K, Aso Y, Hige T, et al. (2016). Direct neural pathways convey distinct visual information to Drosophila mushroom bodies. *eLife* 5:e14009.
17. Zheng Z, Lauritzen JS, Perlman E, et al. (2018). A complete electron microscopy volume of the brain of adult *Drosophila melanogaster*. *Cell* 174(3):730–743.

Appendix A. Reproducing the runs

```
python -m pip install -r requirements.lock.txt
python -m pip install -e .
python -m flyread fetch-data

# main result (recode)
python -m flyread --config configs/odor-recode.yaml    --run-id recode-v2
sweep
python -m flyread --config configs/odor-recode-50.yaml --run-id recode-50-v2
sweep

# growth-matched controls (Sections 4.4-4.5)
python -m flyread --config configs/odor-recode-50-controls.yaml \
    --run-id recode-50-controls sweep
```

```
# reference at the physiological operating point
python -m flyread sweep
python -m flyread --config configs/default.yaml --run-id pctrl-v1 sweep
python -m flyread --config configs/odor-separable.yaml --run-id pctrl-sep sweep
```

The `sweep` command runs one `evaluate` process per seed, resumes completed (seed, condition) pairs from the checkpoint, and merges the shards into a single multi-seed report. Every run writes a manifest. The formal 10-seed run needs ≈ 126 h of single-core compute (≈ 13.8 h wall-clock on 12 cores) and fits in 16 GB; the reported runs are CPU-only with Cython-compiled kernels. The growth-matched control run (9 conditions × 10 seeds) took ≈ 242 h of single-core compute (≈ 28.6 h wall-clock on 10 workers). Re-running with the RNG-isolation fix is the first item of future work; on a smoke configuration the fix makes `trained` and `structural_off` bit-identical.